

제 1장 APNX 통신

1.1 APNX 프로토콜 개요

APNX USB 통신 프로토콜 기반 디바이스 펌웨어 설계

1. 프로젝트 개요

본 프로젝트는 USB Bulk 통신 기반의 APNX 사용자 정의 프로토콜을 설계하고, 이를 마이크로 컨트롤러 펌웨어에 구현한 시스템이다.

- 호스트(PC) -> APNX 프로토콜 프레임 전송
- 디바이스(펌웨어) -> 프레임 검증 및 명령 수행
- 수행 결과 -> ACK 또는 NAK 응답 프레임 반환

펌웨어는 LUFA USB 스택 기반으로 구현되었으며, USB Vendor Bulk Endpoint 를 사용한다.

기반 라이브러리:

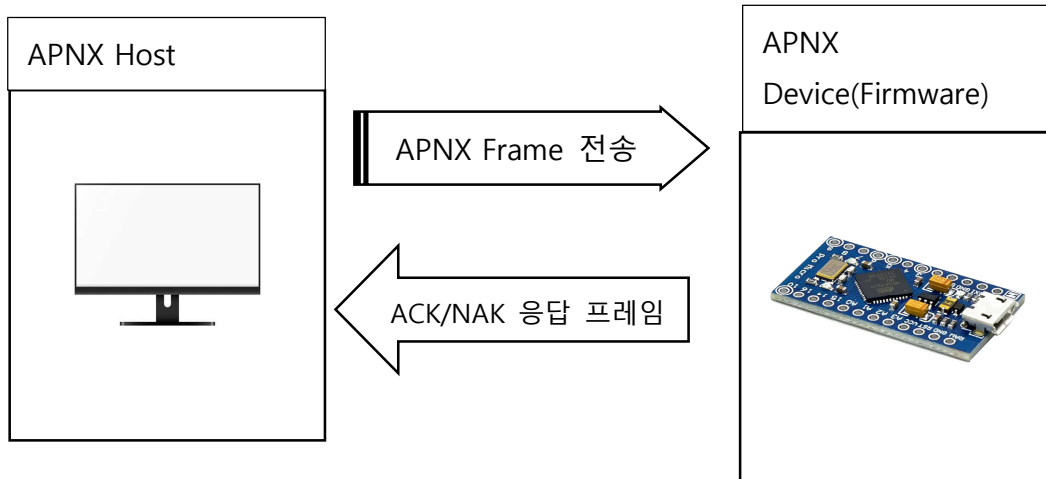
- LUFA

1.2 시스템 구조

1.2.1 통신 모델

본 시스템은 Request-Response 구조로 동작한다.

디바이스는 명령 처리형 임베디드 엔드포인트에 가깝다.



1-3. APNX 프로토콜 프레임 구조

1-3-1. 요구(Request) 프레임 구조

필드	크기	설명
STX	1B	시작 바이트 (0x02)
LEN	1B	전체 패킷 길이
ID	1B	Device ID
CMD	1B	READ / WRITE
CMD_TYPE	1B	주소 체계
DATA_TYPE	1B	8/16/32/64 bit
COUNT	1B	데이터 개수
PAYLOAD	가변	READ : 주소 * Count WRITE : (주소 + 데이터) * Count
CRC	2B	Modbus 방식 CRC

1-3-2. 응답(Request) 프레임

(1) READ 응답

필드	설명
STX	0x02
LEN	STX 포함 전체 길이
ID	Device ID
CMD	ACK_READ
CMD_TYPE	요청과 동일
DATA_TYPE	데이터 타입
COUNT	개수
PAYLOAD	읽은 데이터 : Data * Count
CRC	2B

(2) WRITE 응답

필드	설명
STX	0x02
LEN	고정 길이 (STX 포함 전체 길이)
ID	Device ID
CMD	ACK_WRITE
CMD_TYPE	요청과 동일
CRC	2B

(3) 오류 응답(NAK)

필드	설명
STX	0x02
LEN	고정 길이 (STX 포함 전체 길이)
ID	Device ID
CMD	NAK
CMD_TYPE	요청과 동일
ERROR_CODE	에러 코드(2B)
CRC	2B

1-4. 명령 체계

1-4-1. 지원 명령 (CMD)

명령	값	설명
READ	0	디바이스 메모리 읽기
WRITE	1	디바이스 메모리 쓰기
ACK_READ	5	읽기 성공
ACK_WRITE	6	쓰기 성공
NAK	15	오류 응답

1-4-2. 명령어 타입(CMD_TYPE)

명령 타입	값	설명
Single	1	개별 읽기/쓰기

1-4-2. 데이터 타입

타입	크기
DATA_TYPE_8	1 byte
DATA_TYPE_16	2 bytes
DATA_TYPE_32	4 bytes
DATA_TYPE_64	8 bytes

※모든 전송 데이터는 Big Endian 기반으로 처리되며, 디바이스 내부 Little Endian 구조와 변환을 수행한다.

1-5. 논리 메모리 맵 구조

디바이스는 논리 주소 체계를 사용한다.

영역	시작 주소	크기
INPUT	0x100	16B
OUTPUT	0x200	16B
DATA	0x300	64B
FLAG	0x400	32B

주소 검증은 다음 기준으로 수행된다:

`addr_type = address & 0xF00`

`offset = address & ~addr_type`

※범위를 초과할 경우 NAK 응답을 생성한다.

1-6. 통신 동작 예시

1-6-1. READ 요청

디바이스를 데이터 타입에 맞게 직접 지정하여 읽는 기능이다. 한번에 16 개의 독립된 디바이스 메모리를 읽을 수 있다.

다음은 1 번 국번의 300, 305 16 비트를 읽는 경우의 예시이다.

(이때 주소 0x300 에는 0x1234, 주소 0x305 에는 0x5678 이 들어있다)

APNX 개별읽기 요구 프레임

구분	헤더	총 길이	국번	명령어	명령어 타입	데이터 타입	데이터 수	주소	주소	CRC
프레임	STX	LEN	ID	READ	SINGLE	UINT16	COUNT	ADDRESS	ADDRESS	CRC
Hex	0x02	0x0D	0x01	0x00	0x01	0x01	0x02	0x03, 0x00	0x03, 0x05	0xFF 0xFF

APNX 응답 프레임(ACK 응답 시)

- 0x1234 -> Big Endian 전송 -> 12 34
- 0x5678 -> Big Endian 전송 -> 56 78

구분	헤더	총 길이	국번	명령어	명령어 타입	데이터 타입	데이터 수	데이터 값	데이터 값	CRC
프레임	STX	LEN	ID	ACK_READ	SINGLE	UINT16	COUNT	DATA	DATA	CRC
Hex	0x02	0x0D	0x01	0x05	0x01	0x01	0x02	0x12, 0x34	0x56, 0x78	0xFF 0xFF

APNX 응답 프레임(NAK 응답 시)

구분	헤더	총 길이	국번	명령어	명령어 타입	에러 코드	CRC
프레임	STX	LEN	ID	NAK	SINGLE	에러 코드(2 바이트)	CRC
Hex	0x02	0x0D	0x01	0x0F	0x01	0xFF, 0xFF	0xFF, 0xFF

1-6-2. WRITE 요청

디바이스 메모리를 직접 지정해 메모리 데이터 타입에 맞게 쓰는 기능이다.

다음은 1 번 국번의 300 주소에 0xFF 를 쓰려고 하는 경우의 예시이다.

APNX 개별쓰기 요구 프레임

구분	헤더	총 길이	국번	명령어	명령어 타입	데이터 타입	데이터 수	주소	데이터 값	CRC
프레임	STX	LEN	ID	WRITE	SINGLE	UINT8	COUNT	ADDRESS	DATA	CRC
Hex	0x02	0x0D	0x01	0x01	0x01	0x00	0x01	0x03, 0x00	0xFF	0xFF, 0xFF

APNX 응답 프레임(ACK 응답 시)

구분	헤더	총 길이	국번	명령어	명령어 타입	데이터 타입
프레임	STX	LEN	ID	ACK_WRITE	SINGLE	CRC
Hex	0x02	0x0D	0x01	0x06	0x01	0xFF

APNX 응답 프레임(NAK 응답 시)

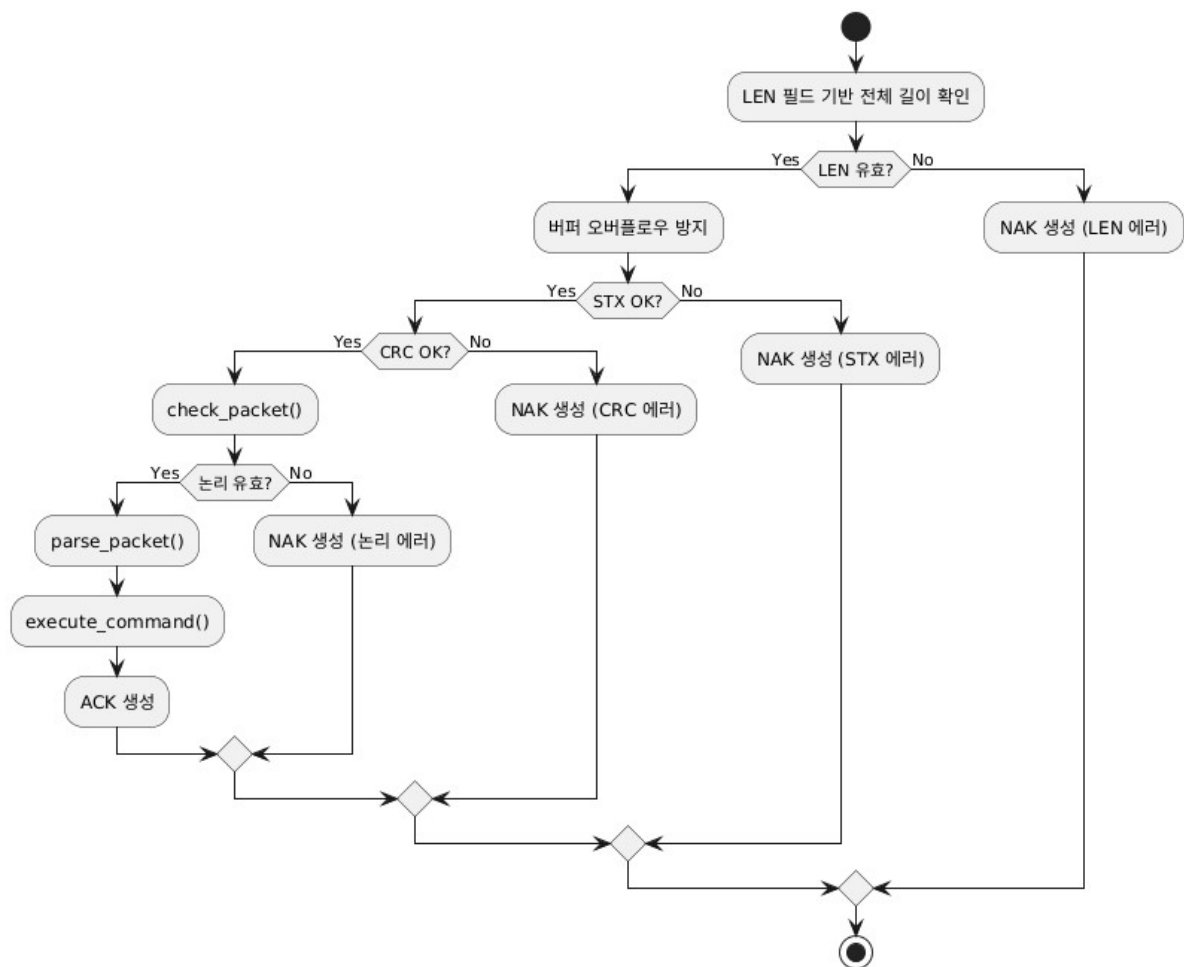
구분	헤더	총 길이	국번	명령어	명령어 타입	에러 코드	CRC
프레임	STX	LEN	ID	NAK	SINGLE	에러 코드(2 바이트)	CRC
Hex	0x02	0x0D	0x01	0x0F	0x01	0xFF, 0xFF	0xFF, 0xFF

1-7. 패킷 처리 구조

1-7-1 수신 상태 머신 (RX FSM)

- expected_len
- copied_len
- buf[MAX_PACKET_LEN]

동작 과정



1-8. 오류 코드 체계

오류 코드는 계층적으로 분류된다.

x00xx : 명령 오류

- ERR_INVALID_CMD
- ERR_INVALID_CMD_TYPE

0x01xx : 데이터 오류

- ERR_INVALID_DATA_TYPE
- ERR_INVALID_COUNT
- ERR_OUT_OF_RANGE

0x02xx : 주소 오류

- ERR_INVALID_ADDRESS

0x03xx: 프레임 검증 에러

- ERR_INVALID_LEN
- ERR_CRC_FAIL
- ERR_INVALID_STX

0xFFxx : 시스템 오류

- ERR_INTERNAL
- ERR_UNKNOWN

1-9. CRC 처리

본 프로토콜의 오류 검출을 위해 Modbus RTU CRC16 알고리즘을 기반으로 한다.

항목	값
Polynomial	0xA001
Initial Value	0xFFFF
Bit Order	LSB First
Final XOR	없음
CRC Length	2 Bytes
Transmission Order	Big Endian (High → Low)

※ 내부 연산은 Modbus RTU와 동일하나, 프레임 전송 시 CRC 바이트 순서는 Big Endian으로 정의한다.

1-9-2. CRC 계산 범위

CRC 는 다음 영역에 대해 계산된다:

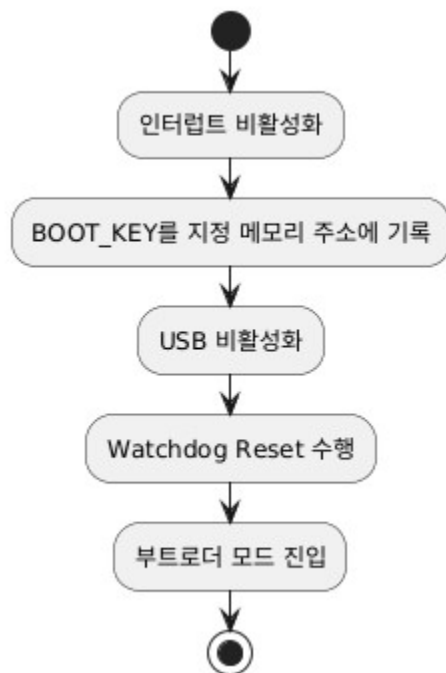
STX + LEN + HEADER + DATA (CRC 필드는 계산에서 제외된다.)

1-10. 부트로더 진입 매커니즘

특수 명령 수신 시:

cmd == GO_TO_BOOT (0x77)

동작 순서



※ 이는 펌웨어 업데이트를 위한 설계이다.